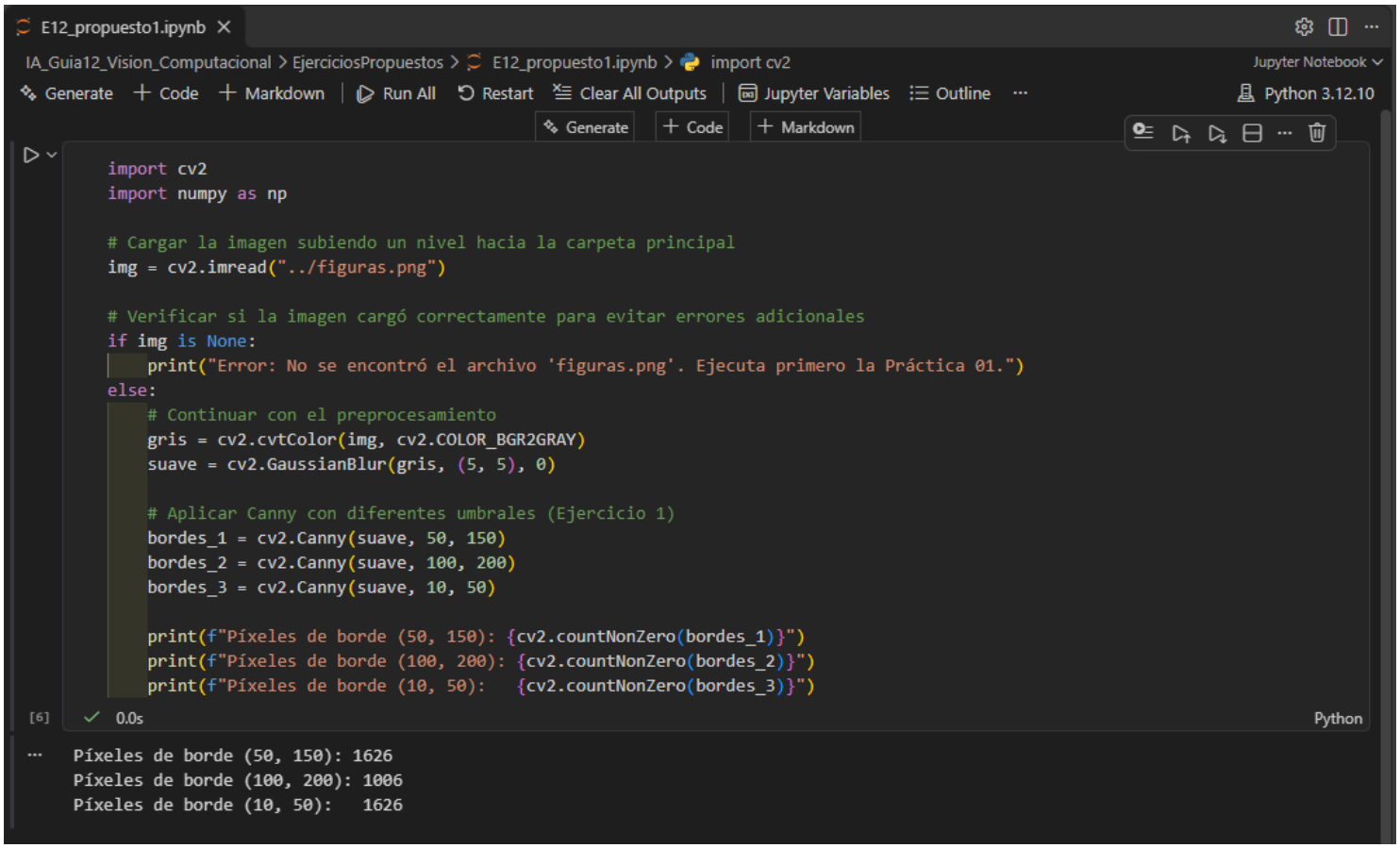
1. En la Práctica 02 cambia los umbrales de Canny de (50, 150) a (100, 200) y a (10, 50). ¿Cuántos píxeles de borde se detectan en cada caso? Explica qué hacen los dos umbrales.



2. Modifica la Práctica 04 para segmentar solo los objetos azules de figuras.png y dibuja una caja delimitadora alrededor de ellos.

E12_propuesto2.ipynb

ia12_Vision_Computacional > EjerciciosPropuestos > E12_propuesto2.ipynb > # 04_segmentacion.py - Segmentar objetos azules y dibujar cajas delimitadoras Jupyter Notebook

Generate | + Code | + Markdown | Interrupt | Restart | Clear All Outputs | Go To | Jupyter Variables | Outline | ... | Python 3.12.10

```python
img = cv2.imread("figuras.png")

if img is None:
    print("Error: No se encontró la imagen figuras.png")
else:
    print("Imagen cargada correctamente")
```

[7] ✓ 0.0s Python

```
Imagen cargada correctamente
```

```python
# 04_segmentacion.py - Segmentar objetos azules y dibujar cajas delimitadoras
import cv2
import numpy as np

img = cv2.imread("figuras.png")

# A) SEGMENTACIÓN POR COLOR: aislar solo el AZUL
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

inferior_azul = np.array([100, 150, 50])
superior_azul = np.array([130, 255, 255])

mascara_azul = cv2.inRange(hsv, inferior_azul, superior_azul)
solo_azul = cv2.bitwise_and(img, img, mask=mascara_azul)

print("Píxeles azules:", cv2.countNonZero(mascara_azul))

# B) ENCONTRAR CONTORNOS Y DIBUJAR CAJAS DELIMITADORAS
contornos, _ = cv2.findContours(
    mascara_azul, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

for c in contornos:
    if cv2.contourArea(c) > 50:
        x, y, w, h = cv2.boundingRect(c)

        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

        cv2.putText(
            img, "Objeto Azul", (x, y - 6),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1
        )

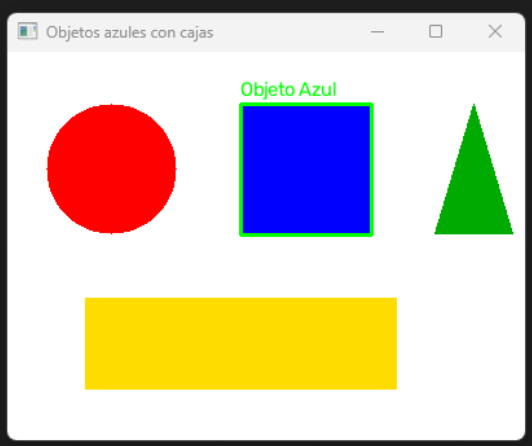
# C) MOSTRAR Y GUARDAR RESULTADOS
cv2.imwrite("segmento_azul.png", solo_azul)
cv2.imwrite("objetos_azules_detectados.png", img)

cv2.imshow("Solo azul", solo_azul)
cv2.imshow("Objetos azules con cajas", img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

[8] 14.9s Python

```
Píxeles azules: 10201
```





3. Agrega un pentágono a la imagen de la Práctica 01 y mejora la función clasificar de la Práctica 03 para reconocerlo.

E12_propuesto3.ipynb

2_Vision_Computacional > EjerciciosPropuestos > E12_propuesto3.ipynb > # Ejercicio 3: Agregar pentágono a figuras.png y reconocerlo en la función clasificar

Jupyter Notebook

Generate | Code | Markdown | Interrupt | Restart | Clear All Outputs | Go To | Jupyter Variables | Outline | Python 3.12.10

```python
# Ejercicio 3: Agregar pentágono a figuras.png y reconocerlo en la función clasificar
import cv2
import numpy as np

# Recreamos la imagen base agregando un pentágono color morado/rosado
img = np.full((300, 400, 3), 255, dtype=np.uint8)
cv2.circle(img, (80, 90), 50, (0, 0, 255), -1)  # Círculo rojo
cv2.rectangle(img, (180, 40), (280, 140), (255, 0, 0), -1)  # Cuadrado azul
triangulo = np.array([[330, 140], [390, 140], [360, 40]])
cv2.fillPoly(img, [triangulo], (0, 170, 0))  # Triángulo verde
cv2.rectangle(img, (60, 190), (300, 260), (0, 220, 255), -1)  # Rectángulo amarillo

# Coordenadas aproximadas para un pentágono
pentagono = np.array([[70, 100], [110, 70], [150, 100], [135, 140], [85, 140]])
# Dibujamos un pentágono (por ejemplo, color celeste/morado: BGR [200, 100, 0])
# Nota: Para que sea detectado por umbral binario invertido (fondo blanco),
# asegurémonos de dibujarlo con un tono oscuro o gris, o adaptemos el umbral.
# Dibujémolo de color gris oscuro para que umbral binario lo detecte:
cv2.fillPoly(img, [pentagono], (100, 100, 100))

# Preprocesamiento y detección
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binaria = cv2.threshold(gris, 240, 255, cv2.THRESH_BINARY_INV)
contornos, _ = cv2.findContours(
    binaria, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

def clasificar_mejorado(c):
    perimetro = cv2.arcLength(c, True)
    vertices = cv2.approxPolyDP(c, 0.03 * perimetro, True)
    n = len(vertices)
    if n == 3:
        return "triangulo"
    elif n == 4:
        x, y, w, h = cv2.boundingRect(c)
        return "cuadrado" if 0.9 <= w / h <= 1.1 else "rectangulo"
    elif n == 5:
        return "pentagono"
    elif n == 6:
        return "hexagono"
    else:
        return "circulo"


for c in contornos:
    if cv2.contourArea(c) > 100:
        x, y, w, h = cv2.boundingRect(c)
        forma = clasificar_mejorado(c)
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 0, 0), 2)
        cv2.putText(
            img, forma, (x, y - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1
        )

cv2.imshow("Reconocimiento con Pentagono", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

[1] 10.1s Python



4. En la Práctica 05 compara la exactitud de KNN, Árbol de decisión y SVM con los mismos datos. ¿Cuál es el mejor clasificador de imágenes de dígitos?

E12_propuesto4.ipynb

mputacional > EjerciciosPropuestos > E12_propuesto4.ipynb > # Ejercicio 4: Comparar la exactitud de KNN, Árbol de decisión y SVM con el dataset de dígitos Jupyter Notebook

Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ... Python 3.12.10

Start Chat to Generate Code (Ctrl+I)

```python
# Ejercicio 4: Comparar la exactitud de KNN, Árbol de decisión y SVM con el dataset de dígitos
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

# Cargar dataset de dígitos
digitos = load_digits()
X = digitos.images.reshape(len(digitos.images), -1)  # Vector de 64 valores
y = digitos.target

# Particionar datos (75% entrenamiento, 25% prueba)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# 1. K-Vecinos más cercanos (KNN)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_tr, y_tr)
pred_knn = knn.predict(X_te)

# 2. Árbol de Decisión
arbol = DecisionTreeClassifier(random_state=42)
arbol.fit(X_tr, y_tr)
pred_arbol = arbol.predict(X_te)

# 3. Máquina de Soporte Vectorial (SVM - de la práctica original)
svm = SVC(gamma=0.001, random_state=42)
svm.fit(X_tr, y_tr)
pred_svm = svm.predict(X_te)

# Imprimir resultados comparativos
print(f"Exactitud KNN (k=3):        {accuracy_score(y_te, pred_knn):.2%}")
print(f"Exactitud Árbol de Decisión: {accuracy_score(y_te, pred_arbol):.2%}")
print(f"Exactitud SVM (SVC):         {accuracy_score(y_te, pred_svm):.2%}")
print(
    "\nConclusión: Por lo general, SVM (Support Vector Machine) suele"
    " obtener la mayor exactitud en tareas de reconocimiento espacial de"
    " imágenes vectorizadas de alta dimensión como los dígitos."
)
```

[1] ✓ 1.0s Python

```
Exactitud KNN (k=3):        98.67%
Exactitud Árbol de Decisión: 85.78%
Exactitud SVM (SVC):         98.89%

Conclusión: Por lo general, SVM (Support Vector Machine) suele obtener la mayor exactitud en tareas de reconocimiento espacial de
```

5. Modifica la Práctica 07 para agregar el modo «rojo» (tecla r), que muestre solo los objetos rojos que aparecen frente a la cámara usando la máscara HSV de la Práctica 04.

